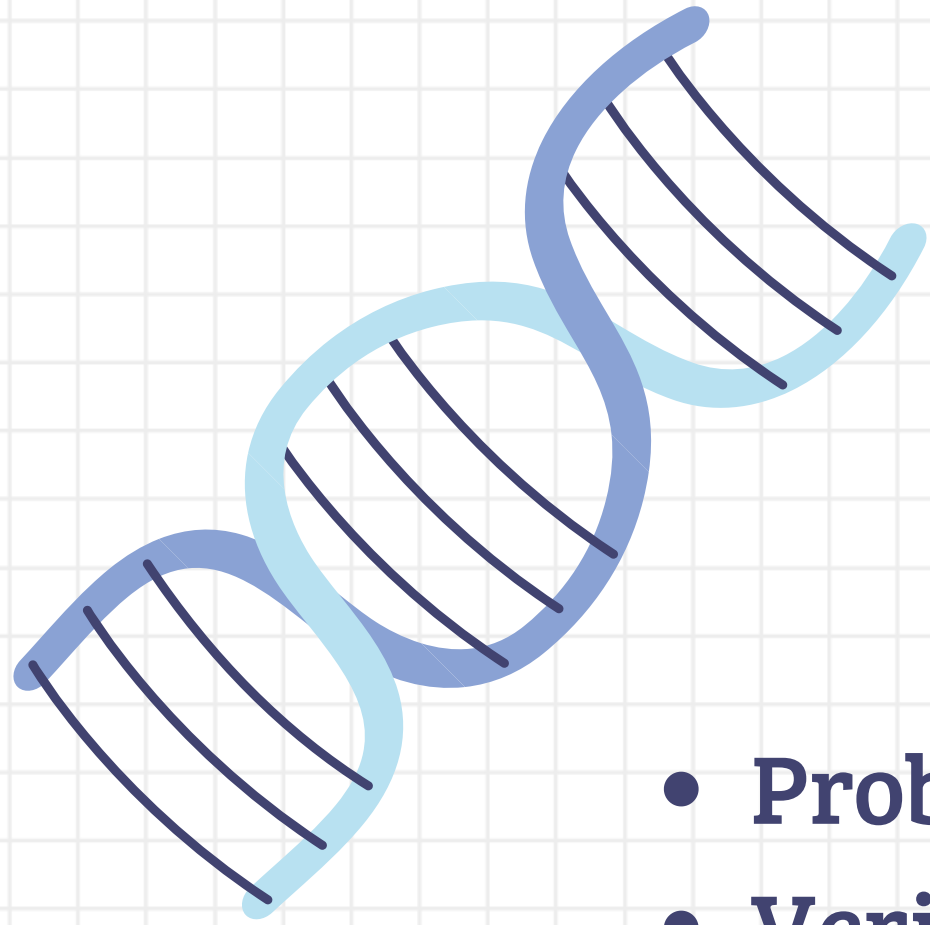


HİTİT ÜNİVERSİTESİ
MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ

Obezite Riskinin Çok Sınıflı Tahmini

MAKİNE ÖĞRENMESİ DERSİ
DOÇ. DR. ÖMER FARUK AKMEŞE

HAZIRLAYANLAR : AYŞE YENİBAĞCI
SHAFİQULLAH QASEMİ
SAFA KAYALAR



SUNUM İÇERİĞİ

- Problem Tanımı ve Veri Seti
- Veri Setimizdeki Değişiklikler
- Eksik Veri Kontrolü
- Veri Ön İşleme ve Eğitim–Test Ayrımı
- Model Karşılaştırmaları
- XGBoost İyileştirmesi
- Overfitting ve Underfitting Durumu
- Streamlit Uygulaması



Obezite, günümüzde dünya genelinde hızla artan ve bireylerin yaşam kalitesini ciddi şekilde etkileyen önemli bir sağlık problemidir. Obezite; kalp hastalıkları, diyabet ve hipertansiyon gibi birçok kronik hastalığın temel nedenlerinden biri olarak kabul edilmektedir. Bu nedenle bireylerin obezite riskinin erken belirlenmesi ve uygun önlemlerin alınması büyük önem taşımaktadır.

Bu çalışmada, bireylerin demografik özellikleri (yaş, cinsiyet), fiziksel ölçümleri (boy, kilo) ve yaşam tarzı alışkanlıkları (beslenme, fiziksel aktivite, su tüketimi, ulaşım şekli vb.) kullanılarak obezite seviyelerinin tahmin edilmesi amaçlanmaktadır. Kullanılan veri setinde bu özellikler detaylı şekilde yer almakta olup hedef değişken olarak bireyin obezite sınıfını gösteren NObeyesdad sütunu bulunmaktadır .

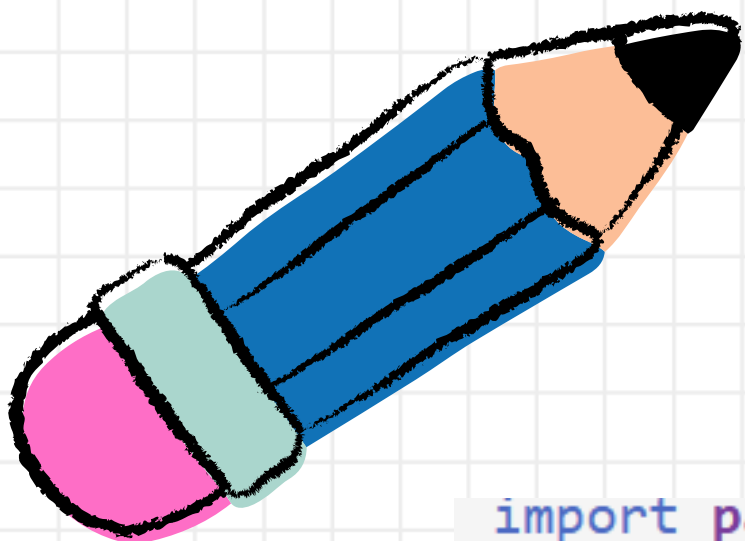


PROJENİN AMACI

Bu projenin temel amacı, bireylerin sahip olduğu çeşitli özellikleri kullanarak obezite seviyelerini doğru bir şekilde tahmin edebilen bir makine öğrenmesi modeli geliştirmektir.

Bu doğrultuda:

- **Obeziteyi etkileyen faktörler analiz edilecektir.**
- **Veri setindeki değişkenler incelenerek önemli özellikler belirlenecektir.**
- **Farklı makine öğrenmesi algoritmaları uygulanacaktır.**
- **Modellerin performansları karşılaştırılarak en iyi sonuç veren yöntem belirlenecektir.**



```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from lazypredict.Supervised import LazyClassifier
from xgboost import XGBClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.model_selection import GridSearchCV
```


Veri Setinin Özellikleri

```
df = pd.read_csv("veri.csv")
kolonlar = {
    "id": "Kayıt numarası",
    "Gender": "Cinsiyet",
    "Age": "Yaş",
    "Height": "Boy",
    "Weight": "Kilo",
    "family_history_with_overweight": "Ailede fazla kilo geçmişi",
    "FAVC": "Yüksek kalorili yiyecek tüketimi",
    "FCVC": "Sebze tüketim sıklığı",
    "NCP": "Günlük ana öğün sayısı",
    "CAEC": "Öğün arası yeme alışkanlığı",
    "SMOKE": "Sigara kullanımı",
    "CH2O": "Günlük su tüketimi",
    "SCC": "Kalori takibi yapıyor mu",
    "FAF": "Fiziksel aktivite sıklığı",
    "TUE": "Teknoloji kullanım süresi",
    "CALC": "Alkol tüketimi",
    "MTRANS": "Ulaşım türü",
    "NObeyesdad": "Obezite seviyesi (hedef değişken)"
}

print(" KOLON AÇIKLAMALARI:\n")
for col in df.columns:
    print(f"{col} ({kolonlar[col]})")
```

```
print("\n Eksik veri kontrolü:")  
print(df.isnull().sum())
```

```
print("\n Veri tipleri:")  
print(df.dtypes)
```

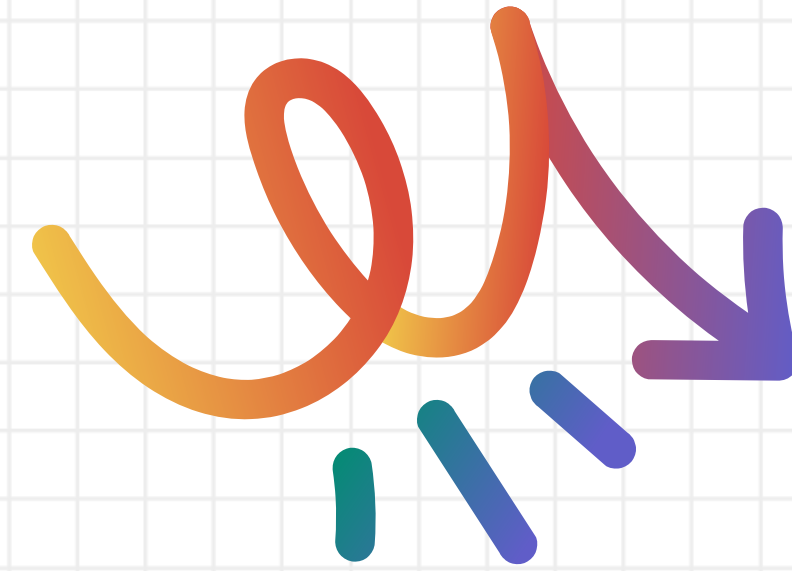
```
Eksik veri kontrolü:  
id          0  
Gender      0  
Age         0  
Height     0  
Weight     0  
family_history_with_overweight  0  
FAVC       0  
FCVC       0  
NCP        0  
CAEC       0  
SMOKE      0  
CH2O       0  
SCC        0  
FAF        0  
TUE        0  
CALC       0  
MTRANS     0  
NObeyesdad 0
```

```
Veri tipleri:  
id          int64  
Gender      object  
Age         float64  
Height     float64  
Weight     float64  
family_history_with_overweight  object  
FAVC       object  
FCVC       float64  
NCP        float64  
CAEC       object  
SMOKE      object  
CH2O       float64  
SCC        object  
FAF        float64  
TUE        float64  
CALC       object  
MTRANS     object  
NObeyesdad  object
```

```
print(df["NObeyesdad"].value_counts())
```

NObeyesdad	
Obesity_Type_III	4046
Obesity_Type_II	3248
Normal_Weight	3082
Obesity_Type_I	2910
Insufficient_Weight	2523
Overweight_Level_II	2522
Overweight_Level_I	2427


```
df["BMI"] = df["Weight"] / (df["Height"] ** 2)
df = df.drop(["id", "Height", "Weight"], axis=1)
```



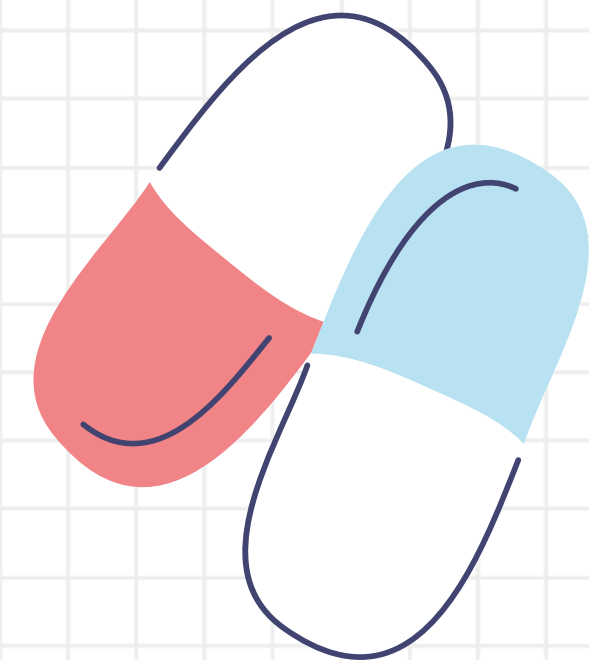
- Veri setinde bulunan id değişkeni, model için anlamlı bir bilgi taşımadığı için kaldırılmıştır.
- Boy (Height) ve Kilo (Weight) değişkenleri kullanılarak Vücut Kitle İndeksi (VKİ) oluşturulmuştur.
- VKİ, bireyin obezite seviyesini daha iyi temsil eden bir ölçüt olduğu için modele yeni bir özellik olarak eklenmiştir.
- Bu nedenle modelin daha sade ve etkili olması amacıyla boy ve kilo değişkenleri veri setinden çıkarılmıştır.

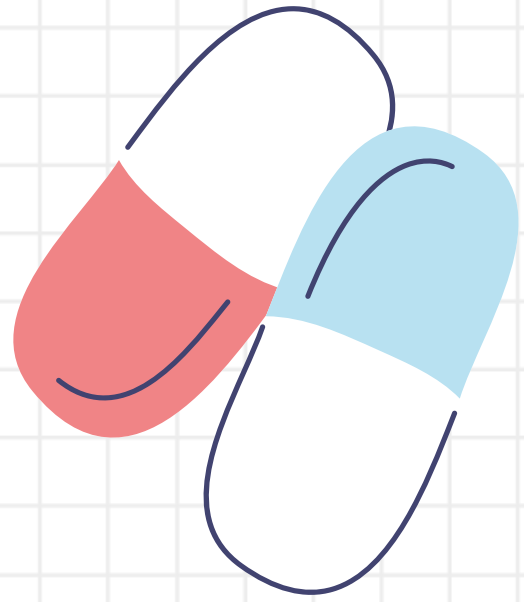
Gender (Cinsiyet)
Age (Yaş)
family_history_with_overweight (Ailede fazla kilo geçmişi)
FAVC (Yüksek kalorili yiyecek tüketimi)
FCVC (Sebze tüketim sıklığı)
NCP (Günlük ana öğün sayısı)
CAEC (Öğün arası yeme alışkanlığı)
SMOKE (Sigara kullanımı)
CH2O (Günlük su tüketimi)
SCC (Kalori takibi yapıyor mu)
FAF (Fiziksel aktivite sıklığı)
TUE (Teknoloji kullanım süresi)
CALC (Alkol tüketimi)
MTRANS (Ulaşım türü)
NObesidad (Obezite seviyesi (hedef değişken))
BMI (vki)

Hedef ve giriş değişkenleri belirlenmiş, kategorik veriler sayısal formata dönüştürülerek modellemeye hazır hale getirilmiştir.

```
Giriş Değişkenleri :  
Gender  
Age  
family_history_with_overweight  
FAVC  
FCVC  
NCP  
CAEC  
SMOKE  
CH2O  
SCC  
FAF  
TUE  
CALC  
MTRANS  
BMI
```

```
y = df["NObeyesdad"]  
X = df.drop("NObeyesdad", axis=1)  
  
print("\n Giriş Değişkenleri :\n")  
for col in X.columns:  
    print(col)  
  
X = pd.get_dummies(X)  
  
le = LabelEncoder()  
y = le.fit_transform(y)
```



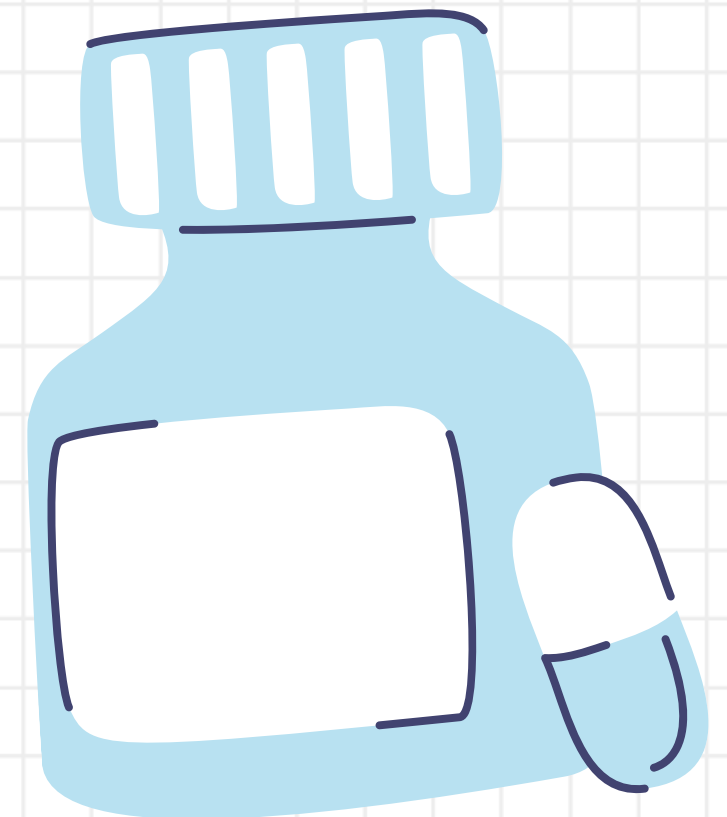


```
print("\n Obezite Seviyeleri\n")
for i, sinif in enumerate(le.classes_):
    print(f"{i} → {sinif}")
```

Obezite Seviyeleri

0 → Insufficient_Weight
1 → Normal_Weight
2 → Obesity_Type_I
3 → Obesity_Type_II
4 → Obesity_Type_III
5 → Overweight_Level_I
6 → Overweight_Level_II

0 → Yetersiz kilo (Zayıf)
1 → Normal kilo
2 → 1. derece obezite
3 → 2. derece obezite
4 → 3. derece obezite
5 → → Fazla kilolu (1. seviye)
6 → Fazla kilolu (2. seviye)



LOJİSTİK REGRESYON VE SONUÇLARI

```
X_egitim, X_test, y_egitim, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

olceklendirici = StandardScaler()
X_egitim = olceklendirici.fit_transform(X_egitim)
X_test = olceklendirici.transform(X_test)

lojistik_model = LogisticRegression(max_iter=1000)
lojistik_model.fit(X_egitim, y_egitim)
y_tahmin = lojistik_model.predict(X_test)

dogruluk = accuracy_score(y_test, y_tahmin)
hassaslik = precision_score(y_test, y_tahmin, average="weighted")
duyarlilik = recall_score(y_test, y_tahmin, average="weighted")
f1 = f1_score(y_test, y_tahmin, average="weighted")

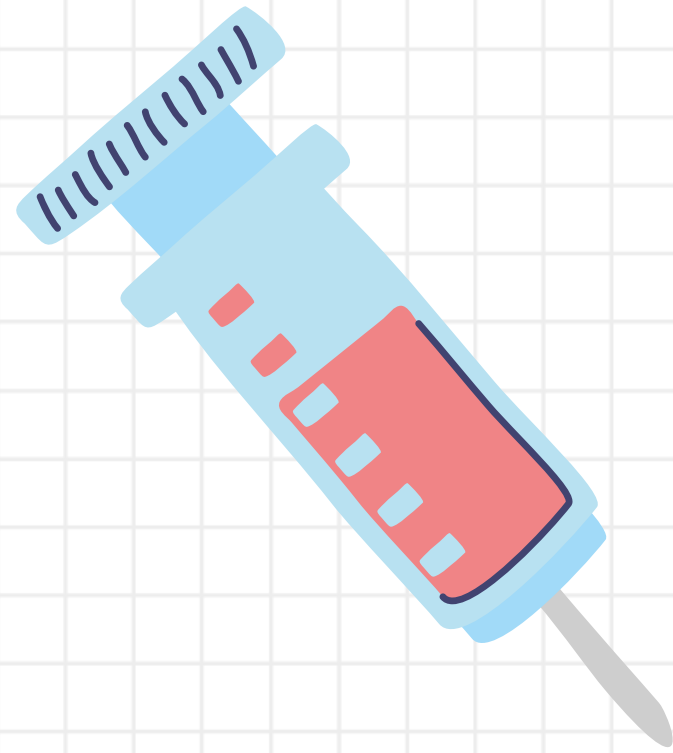
print("\n LOJİSTİK REGRESYON SONUÇLARI")
print("Doğruluk (Accuracy):", dogruluk)
print("Hassaslık (Precision):", hassaslik)
print("Duyarlılık (Recall):", duyarlilik)
print("F1 Skoru:", f1)

print("\n Sınıflandırma Raporu:")
print(classification_report(y_test, y_tahmin))
print("\n Karışıklık Matrisi:")
print(confusion_matrix(y_test, y_tahmin))
```

Modelin daha dengeli öğrenmesi ve performansının artırılması amacıyla veri setine standardizasyon işlemi uygulanmıştır.

LOJİSTİK REGRESYON SONUÇLARI

Doğruluk (Accuracy): 0.8516377649325626
Hassaslık (Precision): 0.850667464663748
Duyarlılık (Recall): 0.8516377649325626
F1 Skoru: 0.8508147057652908



Karışıklık Matrisi:

```
[[489  32   0   0   0   1   2]
 [ 59 500   0   0   0  58   9]
 [  2   0 431  42   4  17  47]
 [  0   0  26 624   0   0   7]
 [  0   0   0   1 802   1   0]
 [  1  47  14   1   0 344  77]
 [  0   8  66  14   0  80 346]]
```



Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.89	0.93	0.91	524
1	0.85	0.80	0.82	626
2	0.80	0.79	0.80	543
3	0.91	0.95	0.93	657
4	1.00	1.00	1.00	804
5	0.69	0.71	0.70	484
6	0.71	0.67	0.69	514
accuracy			0.85	4152
macro avg	0.84	0.84	0.84	4152
weighted avg	0.85	0.85	0.85	4152

RANDOM FOREST VE SONUÇLARI

```
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_egitim, y_egitim)
y_tahmin_rf = rf_model.predict(X_test)

dogruluk = accuracy_score(y_test, y_tahmin_rf)
hassaslik = precision_score(y_test, y_tahmin_rf, average="weighted")
duyarlilik = recall_score(y_test, y_tahmin_rf, average="weighted")
f1 = f1_score(y_test, y_tahmin_rf, average="weighted")

print("\n RANDOM FOREST SONUÇLARI")
print("Doğruluk (Accuracy):", dogruluk)
print("Hassaslık (Precision):", hassaslik)
print("Duyarlılık (Recall):", duyarlilik)
print("F1 Skoru:", f1)

print("\n Sınıflandırma Raporu:")
print(classification_report(y_test, y_tahmin_rf))

print("\n Karışıklık Matrisi:")
print(confusion_matrix(y_test, y_tahmin_rf))
```

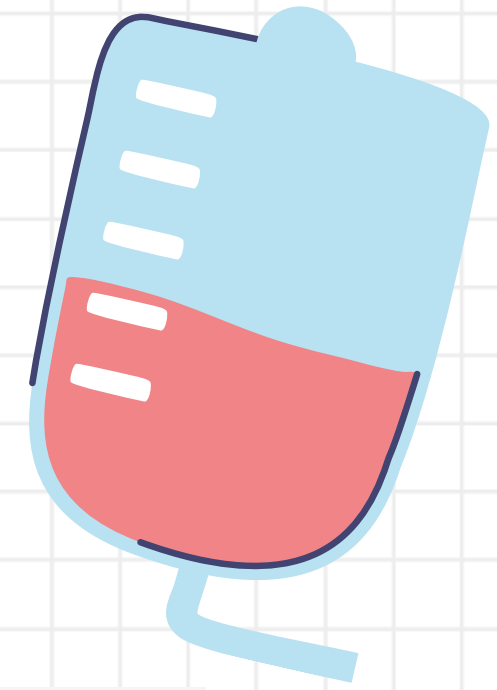

RANDOM FOREST SONUÇLARI

Doğruluk (Accuracy): 0.8716281310211946

Hassaslık (Precision): 0.8717120687982631

Duyarlılık (Recall): 0.8716281310211946

F1 Skoru: 0.8714884136287621



Karışıklık Matrisi:

```
[[478  42   0   0   0   3   1]
 [ 31 545   0   0   0  43   7]
 [  2   3 448  20   2  21  47]
 [  0   0  17 634   0   0   6]
 [  1   0   0   1 802   0   0]
 [  3  58  13   0   0 343  67]
 [  0  15  45   6   0  79 369]]
```

Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.93	0.91	0.92	524
1	0.82	0.87	0.85	626
2	0.86	0.83	0.84	543
3	0.96	0.96	0.96	657
4	1.00	1.00	1.00	804
5	0.70	0.71	0.71	484
6	0.74	0.72	0.73	514
accuracy			0.87	4152
macro avg	0.86	0.86	0.86	4152
weighted avg	0.87	0.87	0.87	4152

KNN VE SONUÇLARI

```
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_egitim, y_egitim)
y_tahmin_knn = knn_model.predict(X_test)
dogruluk = accuracy_score(y_test, y_tahmin_knn)
hassaslik = precision_score(y_test, y_tahmin_knn, average="weighted")
duyarlilik = recall_score(y_test, y_tahmin_knn, average="weighted")
f1 = f1_score(y_test, y_tahmin_knn, average="weighted")

print("\n KNN SONUÇLARI")
print("Doğruluk (Accuracy):", dogruluk)
print("Hassaslık (Precision):", hassaslik)
print("Duyarlılık (Recall):", duyarlilik)
print("F1 Skoru:", f1)

print("\nSınıflandırma Raporu:")
print(classification_report(y_test, y_tahmin_knn))

print("\nKarışıklık Matrisi:")
print(confusion_matrix(y_test, y_tahmin_knn))
```

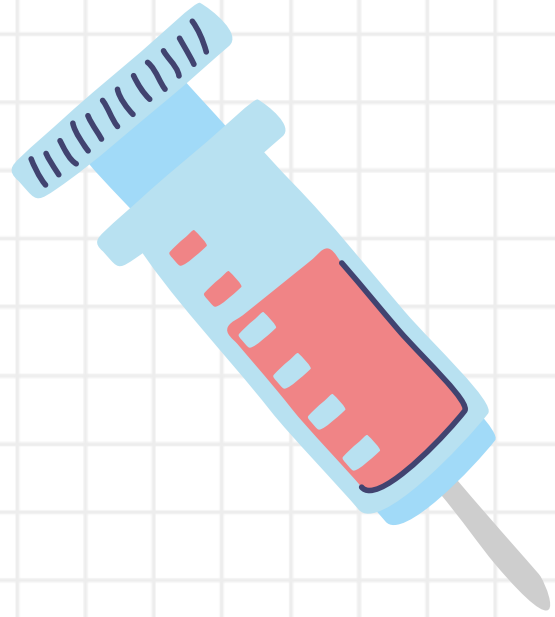
KNN SONUÇLARI

Doğruluk (Accuracy): 0.7360308285163777

Hassaslık (Precision): 0.7343673407600501

Duyarlılık (Recall): 0.7360308285163777

F1 Skoru: 0.733738001160488

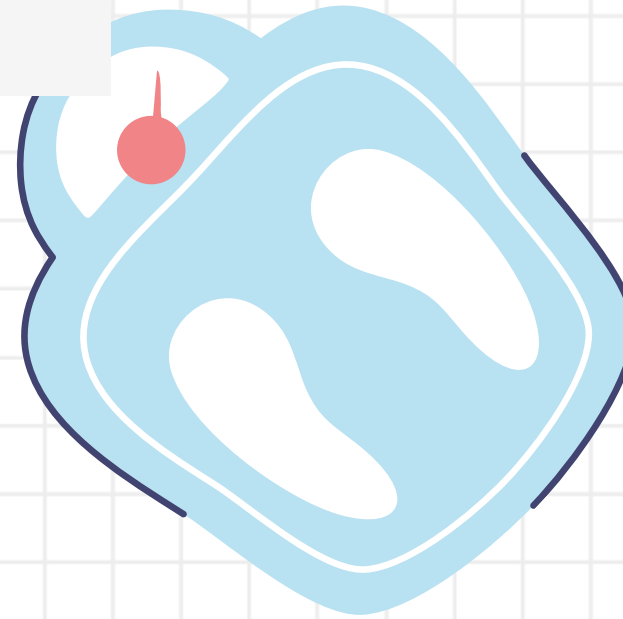


Karışıklık Matrisi:

```
[[397 110  4  0  0  6  7]
 [106 397 11  1  0 70 41]
 [  6  25 380 25  2 34 71]
 [  0  5  33 605  0  1 13]
 [  0  1  0  1 801  0  1]
 [ 31 123 55  1  0 213 61]
 [ 11  66 78 29  0  67 263]]
```

Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.72	0.76	0.74	524
1	0.55	0.63	0.59	626
2	0.68	0.70	0.69	543
3	0.91	0.92	0.92	657
4	1.00	1.00	1.00	804
5	0.54	0.44	0.49	484
6	0.58	0.51	0.54	514
accuracy			0.74	4152
macro avg	0.71	0.71	0.71	4152
weighted avg	0.73	0.74	0.73	4152



SVM VE SONUÇLARI

```
svm_model = SVC(kernel="rbf", random_state=42)
svm_model.fit(X_egitim, y_egitim)
y_tahmin_svm = svm_model.predict(X_test)
dogruluk = accuracy_score(y_test, y_tahmin_svm)
hassaslik = precision_score(y_test, y_tahmin_svm, average="weighted")
duyarlilik = recall_score(y_test, y_tahmin_svm, average="weighted")
f1 = f1_score(y_test, y_tahmin_svm, average="weighted")

print("\n SVM SONUÇLARI")
print("Doğruluk (Accuracy):", dogruluk)
print("Hassaslık (Precision):", hassaslik)
print("Duyarlılık (Recall):", duyarlilik)
print("F1 Skoru:", f1)

print("\nSınıflandırma Raporu:")
print(classification_report(y_test, y_tahmin_svm))

print("\nKarışıklık Matrisi:")
print(confusion_matrix(y_test, y_tahmin_svm))
```

SVM SONUÇLARI

Doğruluk (Accuracy): 0.8357418111753372

Hassaslık (Precision): 0.8364578060418454

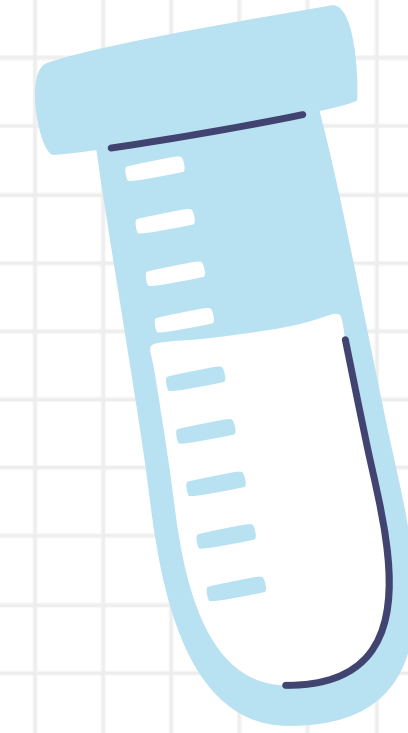
Duyarlılık (Recall): 0.8357418111753372

F1 Skoru: 0.8358038640672637



Karışıklık Matrisi:

```
[[464  57   0   0   0   2   1]
 [ 51 490   0   0   0  65  20]
 [  1   6 423  29   2  24  58]
 [  0   2  11 635   0   0   9]
 [  0   1   1   1 800   1   0]
 [  4  70  12   0   0 312  86]
 [  0  23  49  12   0  84 346]]
```



Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.89	0.89	0.89	524
1	0.76	0.78	0.77	626
2	0.85	0.78	0.81	543
3	0.94	0.97	0.95	657
4	1.00	1.00	1.00	804
5	0.64	0.64	0.64	484
6	0.67	0.67	0.67	514
accuracy			0.84	4152
macro avg	0.82	0.82	0.82	4152
weighted avg	0.84	0.84	0.84	4152

KARAR AĞAÇLARI VE SONUÇLARI

```
karar_agaci_model = DecisionTreeClassifier(random_state=42)
karar_agaci_model.fit(X_egitim, y_egitim)
y_tahmin_karar_agaci = karar_agaci_model.predict(X_test)
dogruluk = accuracy_score(y_test, y_tahmin_karar_agaci)
hassaslik = precision_score(y_test, y_tahmin_karar_agaci, average="weighted")
duyarlilik = recall_score(y_test, y_tahmin_karar_agaci, average="weighted")
f1 = f1_score(y_test, y_tahmin_karar_agaci, average="weighted")

print("\n KARAR AĞACI SONUÇLARI")
print("Doğruluk (Accuracy):", dogruluk)
print("Hassaslık (Precision):", hassaslik)
print("Duyarlılık (Recall):", duyarlilik)
print("F1 Skoru:", f1)

print("\nSınıflandırma Raporu:")
print(classification_report(y_test, y_tahmin_karar_agaci))

print("\nKarışıklık Matrisi:")
print(confusion_matrix(y_test, y_tahmin_karar_agaci))
```

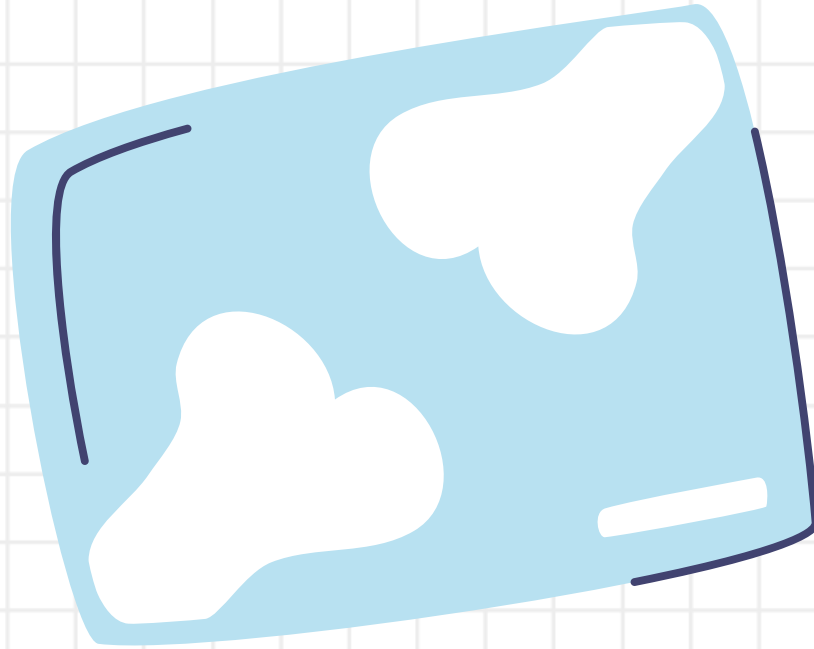

KARAR AĞACI SONUÇLARI

Doğruluk (Accuracy): 0.8140655105973025

Hassaslık (Precision): 0.8157699194887126

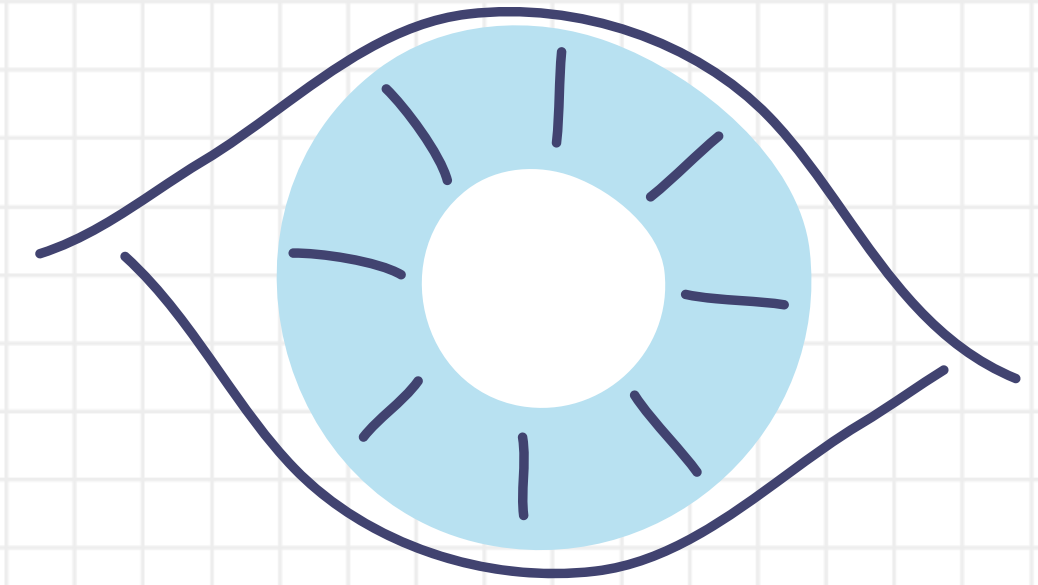
Duyarlılık (Recall): 0.8140655105973025

F1 Skoru: 0.8146870608395576



Karışıklık Matrisi:

```
[[449  62   0   0   0  11   2]
 [ 52 474   3   0   0  81  16]
 [  2   4 419  35   4  34  45]
 [  0   0  44 602   2   0   9]
 [  0   0   0   2 802   0   0]
 [  3  58  30   1   4 303  85]
 [  1  19  51  13   0  99 331]]
```



Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.89	0.86	0.87	524
1	0.77	0.76	0.76	626
2	0.77	0.77	0.77	543
3	0.92	0.92	0.92	657
4	0.99	1.00	0.99	804
5	0.57	0.63	0.60	484
6	0.68	0.64	0.66	514
accuracy			0.81	4152
macro avg	0.80	0.80	0.80	4152
weighted avg	0.82	0.81	0.81	4152

Underfitting Ya Da Overfitting Durumu Var Mı?

```
print("\n OVERFITTING / UNDERFITTING KONTROLÜ\n")
print("Lojistik Regresyon")
print("Train:", lojistik_model.score(X_egitim, y_egitim))
print("Test :", lojistik_model.score(X_test, y_test))
print("\nKNN")
print("Train:", knn_model.score(X_egitim, y_egitim))
print("Test :", knn_model.score(X_test, y_test))
print("\nSVM")
print("Train:", svm_model.score(X_egitim, y_egitim))
print("Test :", svm_model.score(X_test, y_test))
print("\nRandom Forest")
print("Train:", rf_model.score(X_egitim, y_egitim))
print("Test :", rf_model.score(X_test, y_test))
print("\nKarar Ağacı")
print("Train:", karar_agaci_model.score(X_egitim, y_egitim))
print("Test :", karar_agaci_model.score(X_test, y_test))
```

Lojistik Regresyon

Train: 0.8530049379742262

Test : 0.8516377649325626

KNN

Train: 0.818860652776105

Test : 0.7360308285163777

SVM

Train: 0.8646272431651211

Test : 0.8357418111753372

Random Forest

Train: 1.0

Test : 0.8716281310211946

Karar Ağacı

Train: 1.0

Test : 0.8140655105973025

Lojistik Regresyon dengelidir.

KNN overfitting var.

SVM dengelidir.

Random Forest overfitting var.

Karar Ağacı overfitting var.





```
clf = LazyClassifier(verbose=0, ignore_warnings=True)
modeller, tahminler = clf.fit(X_egitim, X_test, y_egitim, y_test)
print(modeller)
```

LazyPredict ile Modellerin Karşılaştırılması

En İyi 5 Model:

1-XGBoost

2-Random Forest

3-Bagging

4-Lojistik Regresyon

5-Extra Tree

Model	Accuracy	Balanced Accuracy	ROC AUC	F1 Score	Precision	Recall	Time Taken
XGBClassifier	0.885597	0.872776	0.986696	0.885491	0.885508	0.885597	3.290762
RandomForestClassifier	0.871628	0.856707	0.981878	0.871488	0.871712	0.871628	2.612185
BaggingClassifier	0.861753	0.847164	0.968841	0.861632	0.861784	0.861753	1.167919
LogisticRegression	0.851879	0.836870	0.979505	0.851046	0.850908	0.851879	0.635882
ExtraTreesClassifier	0.837669	0.819205	0.974587	0.837436	0.837713	0.837669	2.159456
SVC	0.835742	0.818081	NaN	0.835804	0.836458	0.835742	11.125782
LinearDiscriminantAnalysis	0.824904	0.809042	0.974789	0.823703	0.823880	0.824904	0.291294
DecisionTreeClassifier	0.814066	0.795643	0.892957	0.814687	0.815770	0.814066	0.225946
ExtraTreeClassifier	0.770954	0.748761	0.868139	0.771853	0.772923	0.770954	0.106346
NuSVC	0.768304	0.745284	NaN	0.764128	0.764827	0.768304	38.059336
AdaBoostClassifier	0.762524	0.743280	0.952161	0.761309	0.790384	0.762524	1.139366
CalibratedClassifierCV	0.748314	0.719034	0.945665	0.738717	0.738945	0.748314	2.164955
LinearSVC	0.742775	0.712248	NaN	0.729752	0.734358	0.742775	0.608001
KNeighborsClassifier	0.736272	0.708940	0.919141	0.734005	0.734616	0.736272	0.592175
LabelSpreading	0.724952	0.697810	NaN	0.725364	0.726092	0.724952	26.551964
LabelPropagation	0.723748	0.696594	NaN	0.724256	0.725074	0.723748	15.436203
SGDClassifier	0.706888	0.677343	NaN	0.698350	0.694062	0.706888	0.482612
BernoulliNB	0.685934	0.658216	0.931987	0.683948	0.687060	0.685934	0.087940
Perceptron	0.650530	0.619591	NaN	0.653039	0.662283	0.650530	0.116045
PassiveAggressiveClassifier	0.647881	0.610652	NaN	0.645579	0.671068	0.647881	0.147513
GaussianNB	0.635597	0.601713	0.921677	0.606872	0.642634	0.635597	0.076186
RidgeClassifierCV	0.631985	0.594093	NaN	0.596694	0.628645	0.631985	0.654166
RidgeClassifier	0.631985	0.594093	NaN	0.596694	0.628645	0.631985	0.082590
RidgeClassifierCV	0.631985	0.594093	NaN	0.596694	0.628645	0.631985	0.654166
RidgeClassifier	0.631985	0.594093	NaN	0.596694	0.628645	0.631985	0.082590
QuadraticDiscriminantAnalysis	0.627649	0.590801	0.916599	0.611442	0.620085	0.627649	0.142022
NearestCentroid	0.614884	0.580320	0.965736	0.594089	0.602128	0.614884	0.080384
DummyClassifier	0.193642	0.142857	0.500000	0.062828	0.037497	0.193642	0.047893

XGBOOST VE SONUÇLARI

```
xgb_model = XGBClassifier(  
    random_state=42,  
    use_label_encoder=False,  
    eval_metric="mlogloss"  
)  
  
xgb_model.fit(X_egitim, y_egitim)  
y_tahmin_xgb = xgb_model.predict(X_test)  
  
dogruluk = accuracy_score(y_test, y_tahmin_xgb)  
hassaslik = precision_score(y_test, y_tahmin_xgb, average="weighted")  
duyarlilik = recall_score(y_test, y_tahmin_xgb, average="weighted")  
f1 = f1_score(y_test, y_tahmin_xgb, average="weighted")  
  
print("\n XGBOOST SONUÇLARI")  
print("Doğruluk (Accuracy):", dogruluk)  
print("Hassaslık (Precision):", hassaslik)  
print("Duyarlılık (Recall):", duyarlilik)  
print("F1 Skoru:", f1)  
  
print("\n Sınıflandırma Raporu:")  
print(classification_report(y_test, y_tahmin_xgb))  
  
print("\n Karışıklık Matrisi:")  
print(confusion_matrix(y_test, y_tahmin_xgb))
```

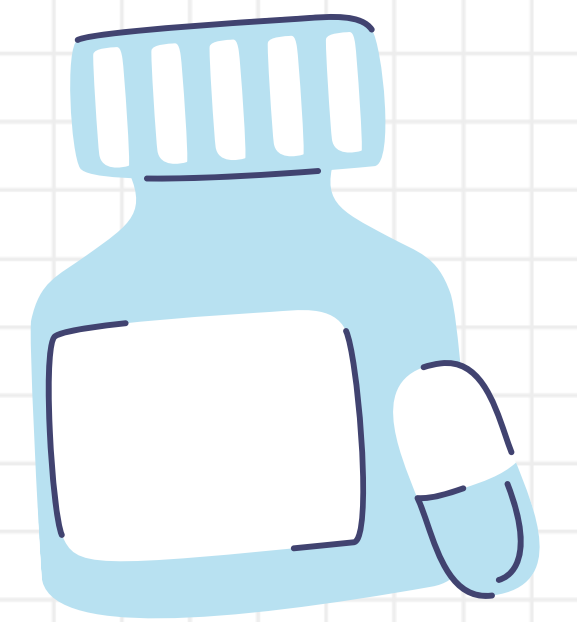
XGBOOST SONUÇLARI

Doğruluk (Accuracy): 0.8855973025048169

Hassaslık (Precision): 0.8855082714132232

Duyarlılık (Recall): 0.8855973025048169

F1 Skoru: 0.8854910734560627

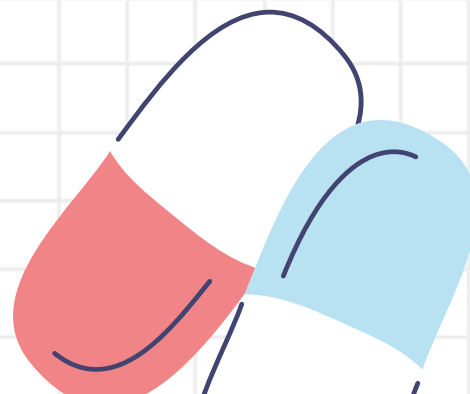


Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.93	0.93	0.93	524
1	0.86	0.87	0.87	626
2	0.86	0.84	0.85	543
3	0.96	0.97	0.96	657
4	1.00	1.00	1.00	804
5	0.74	0.75	0.74	484
6	0.77	0.75	0.76	514
accuracy			0.89	4152
macro avg	0.87	0.87	0.87	4152
weighted avg	0.89	0.89	0.89	4152

Karışıklık Matrisi:

```
[[486  33   0   0   0   4   1]
 [ 32 547   1   0   0  40   6]
 [  2   4 454  18   2  18  45]
 [  0   0  12 637   2   0   6]
 [  0   0   0   1 802   1   0]
 [  1  43  17   0   0 364  59]
 [  0  10  42   8   0  67 387]]
```



BAGGING VE SONUÇLARI

```
bagging_model = BaggingClassifier(random_state=42)
bagging_model.fit(X_egitim, y_egitim)
y_tahmin_bag = bagging_model.predict(X_test)
dogruluk = accuracy_score(y_test, y_tahmin_bag)
hassaslik = precision_score(y_test, y_tahmin_bag, average="weighted")
duyarlilik = recall_score(y_test, y_tahmin_bag, average="weighted")
f1 = f1_score(y_test, y_tahmin_bag, average="weighted")

print("\n BAGGING SONUÇLARI")
print("Doğruluk (Accuracy):", dogruluk)
print("Hassaslık (Precision):", hassaslik)
print("Duyarlılık (Recall):", duyarlilik)
print("F1 Skoru:", f1)

print("\nSınıflandırma Raporu:")
print(classification_report(y_test, y_tahmin_bag))

print("\nKarışıklık Matrisi:")
print(confusion_matrix(y_test, y_tahmin_bag))
```

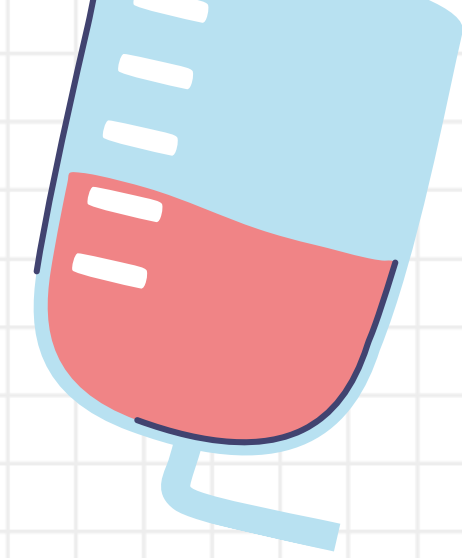

BAGGING SONUÇLARI

Doğruluk (Accuracy): 0.8617533718689788

Hassaslık (Precision): 0.8617842681966358

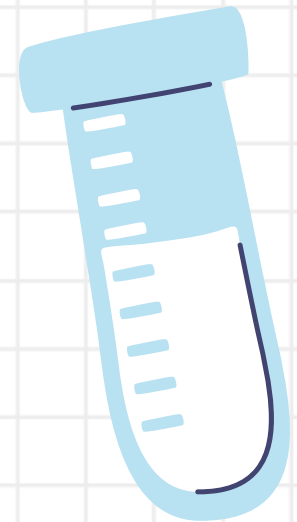
Duyarlılık (Recall): 0.8617533718689788

F1 Skoru: 0.8616315768818629



Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.90	0.93	0.91	524
1	0.83	0.84	0.83	626
2	0.82	0.82	0.82	543
3	0.95	0.95	0.95	657
4	1.00	1.00	1.00	804
5	0.67	0.69	0.68	484
6	0.76	0.71	0.73	514
accuracy			0.86	4152
macro avg	0.85	0.85	0.85	4152
weighted avg	0.86	0.86	0.86	4152



Karışıklık Matrisi:

```
[[485  36   0   0   0   2   1]
 [ 50 523   1   0   0  48   4]
 [   2   2 446  22   2  27  42]
 [   0   0  32 621   0   0   4]
 [   0   0   2   1 801   0   0]
 [   1  59  21   0   1 336  66]
 [   0  10  40   9   0  89 366]]
```

EXTRA TREE VE SONUÇLARI

```
extra_model = ExtraTreesClassifier(random_state=42)
extra_model.fit(X_egitim, y_egitim)
y_tahmin_extra = extra_model.predict(X_test)

dogruluk = accuracy_score(y_test, y_tahmin_extra)
hassaslik = precision_score(y_test, y_tahmin_extra, average="weighted")
duyarlilik = recall_score(y_test, y_tahmin_extra, average="weighted")
f1 = f1_score(y_test, y_tahmin_extra, average="weighted")

print("\n EXTRA TREES SONUÇLARI")
print("Doğruluk (Accuracy):", dogruluk)
print("Hassaslık (Precision):", hassaslik)
print("Duyarlılık (Recall):", duyarlilik)
print("F1 Skoru:", f1)

print("\nSınıflandırma Raporu:")
print(classification_report(y_test, y_tahmin_extra))

print("\nKarışıklık Matrisi:")
print(confusion_matrix(y_test, y_tahmin_extra))
```

EXTRA TREES SONUÇLARI

Doğruluk (Accuracy): 0.8376685934489403

Hassaslık (Precision): 0.8377134691092071

Duyarlılık (Recall): 0.8376685934489403

F1 Skoru: 0.8374363611298901

Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.91	0.87	0.89	524
1	0.76	0.81	0.78	626
2	0.82	0.79	0.81	543
3	0.94	0.96	0.95	657
4	1.00	1.00	1.00	804
5	0.63	0.63	0.63	484
6	0.69	0.66	0.68	514
accuracy			0.84	4152
macro avg	0.82	0.82	0.82	4152
weighted avg	0.84	0.84	0.84	4152



Karışıklık Matrisi:

```
[[458  61   0   0   0   4   1]
 [ 42 509   4   0   0  52  19]
 [   3   7 430  25   2  29  47]
 [   0   1  14 633   0   0   9]
 [   0   0   0   1 801   1   1]
 [   3  73  25   0   0 306  77]
 [   0  20  51  11   0  91 341]]
```

XGBOOST İYİLEŞTİRME VE SONUÇLARI

```
xgb_parametreler = {
    "n_estimators": [100, 200],
    "max_depth": [3, 5, 7],
    "learning_rate": [0.01, 0.05, 0.1],
    "subsample": [0.8, 1.0],
    "colsample_bytree": [0.8, 1.0]
}
xgb_grid = GridSearchCV(
    estimator=XGBClassifier(
        random_state=42,
        eval_metric="mlogloss"
    ),
    param_grid=xgb_parametreler,
    cv=5,
    scoring="f1_weighted",
    n_jobs=-1
)
xgb_grid.fit(X_egitim, y_egitim)
en_ iyi_xgb = xgb_grid.best_estimator_
y_tahmin_xgb_tuned = en_ iyi_xgb.predict(X_test)
print("\n XGBOOST İYİLEŞTİRİLMİŞ SONUÇLAR")
print("Train Accuracy:", en_ iyi_xgb.score(X_egitim, y_egitim))
print("Test Accuracy :", en_ iyi_xgb.score(X_test, y_test))
print("\nAccuracy:", accuracy_score(y_test, y_tahmin_xgb_tuned))
print("Precision:", precision_score(y_test, y_tahmin_xgb_tuned, average="weighted"))
print("Recall:", recall_score(y_test, y_tahmin_xgb_tuned, average="weighted"))
print("F1 Score:", f1_score(y_test, y_tahmin_xgb_tuned, average="weighted"))
print("\nSınıflandırma Raporu:")
print(classification_report(y_test, y_tahmin_xgb_tuned))
print("\nKarışıklık Matrisi:")
print(confusion_matrix(y_test, y_tahmin_xgb_tuned))
```

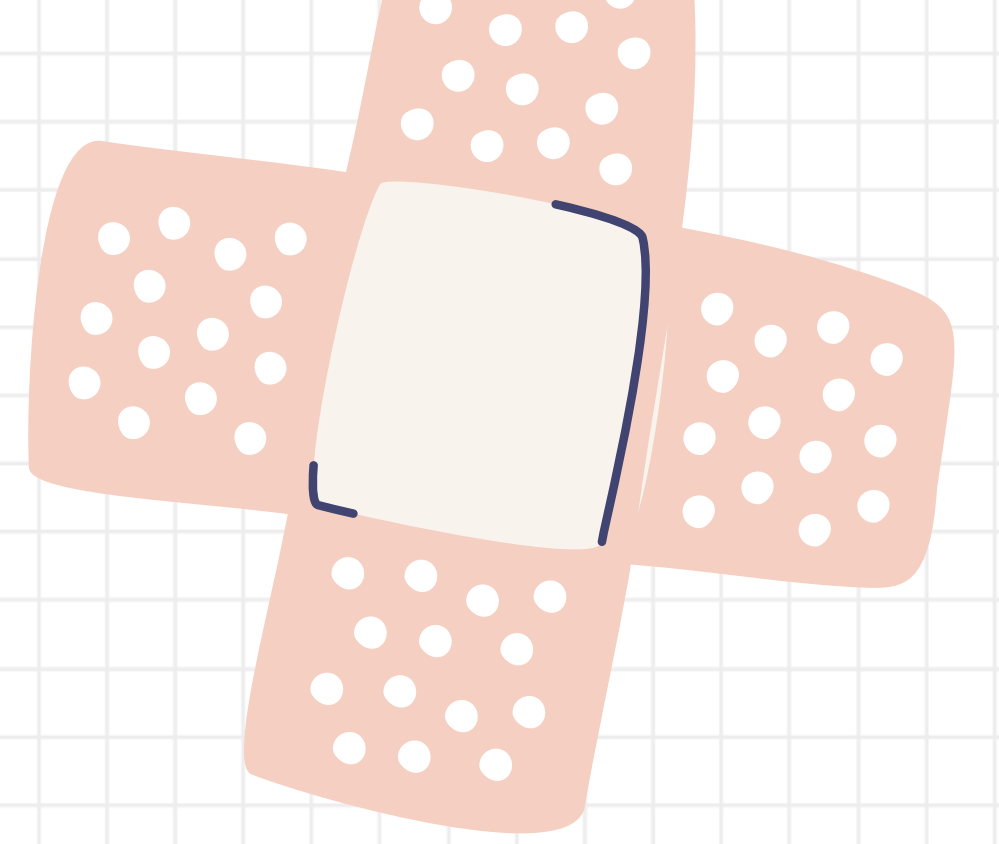
XGBOOST İYİLEŞTİRİLMİŞ SONUÇLAR
Train Accuracy: 0.9429122004094905
Test Accuracy : 0.88752408477842

Accuracy: 0.88752408477842
Precision: 0.8878210494265871
Recall: 0.88752408477842
F1 Score: 0.8876101086273238

Sınıflandırma Raporu:				
	precision	recall	f1-score	support
0	0.93	0.92	0.93	524
1	0.86	0.88	0.87	626
2	0.87	0.84	0.86	543
3	0.97	0.97	0.97	657
4	0.99	1.00	1.00	804
5	0.74	0.75	0.74	484
6	0.77	0.77	0.77	514
accuracy			0.89	4152
macro avg	0.88	0.88	0.88	4152
weighted avg	0.89	0.89	0.89	4152

Karışıklık Matrisi:

```
[[482  37   0   0   0   4   1]
 [ 32 550   1   0   0  36   7]
 [  2   3 457  15   2  23  41]
 [  0   0  14 635   3   0   5]
 [  0   0   1   1 801   1   0]
 [  1  41  14   0   0 362  66]
 [  0  10  37   7   0  62 398]]
```

- **XGBoost başlangıçta en iyi model olduğu için seçildi.**
- **Hiperparametre iyileştirmesi sonrası büyük değişim görülmedi çünkü model zaten optimuma yakındı.**
- **Performansı sınırlayan veri ve özellik yapısıdır.**
- **Daha zayıf modeller teorik olarak geliştirilebilir ancak XGBoost'u geçmeleri genellikle zordur**



**Thank
you!**

